arXiv:2611.10649v1 [cs.LG] 7 Nov 2026

HUAWEI Computing Systems Lab

HUAWEI

🐒 MACaQ: Fast, Model Agnostic Calibrated Quantization for Low-Precision Weights and Activations

Lorenz K. Muller **Philippe Bich** **Chiara Boretti** **Hyun-Min Chang** **Jiawei Zhuang** **Lukas Cavigelli**

Computing Systems Lab, Huawei Technologies, Switzerland.

Weight-activation quantization can reduce the memory and compute cost of neural-network inference, but recent methods often require expensive optimization and architecture-specific propagation of basis transforms across layers. We introduce MACaQ, an MXFP4 W4A4 post-training quantization method that treats every linear layer independently. MACaQ applies a per-channel scale and a block-Hadamard transform online to each layer's activations, while absorbing the corresponding inverse transforms into its weights. Retaining these transforms online avoids cross-layer dependencies and enables a simple streaming implementation of the quantization process, in which only one layer must reside in memory at a time. To improve over round-to-nearest quantization, we introduce Flip-Refinement, a Hessian-aware coordinate-descent procedure that greedily moves individual FP4 codes to adjacent representable values. Despite executing the activation transforms at inference time, MACaQ adds only 2.3% end-to-end overhead over pure MXFP4 on Qwen3-32B in vLLM. MACaQ reaches lower perplexity than DuQuant++ and MR-GPTQ, and already matches their best perplexity on Llama-3-8B in 11× less quantization time. We further apply MACaQ without architecture-specific modifications to an image diffusion model, illustrating the generality of its layer-local design and show quantization of a 122B parameter model in 64 minutes, using only 32.5GB GPU RAM.

Introduction

Perhaps the central idea in recent neural network quantization strategies is the change of basis. To compute the matrix product between an input and a weight matrix, we may transform the input by a matrix P and apply the corresponding inverse transform to the weights, leaving the product unchanged. If P is chosen wisely, the transformed inputs and weights can be substantially easier to quantize.

Two types of transform are widely used to ease quantization. First, activations and weights may be rotated into a different basis using an orthogonal matrix M_{rot}. The Hadamard transform is especially popular because it can be applied in $O(N \log N)$ operations and tends to produce approximately Gaussian-distributed outputs (4). Second, individual channels may be rescaled using a diagonal matrix M_{diag} (29, 18, 21), thereby suppressing outlying channels and balancing the quantization difficulty of activations and weights.

One potential downside of such a change of basis is that, although the weight-side transform can be absorbed into the weights at quantization time, the corresponding input transform ostensibly needs to be applied online. Prior work often avoids this cost by propagating the input transform through the network and absorbing it into preceding layers. This approach is widely used because it can eliminate the explicit runtime transform (1). However, it also introduces dependencies between layers: applying the transform to one linear layer may require modifying its normalization parameters, preceding projections, residual branches, or other connected operators.

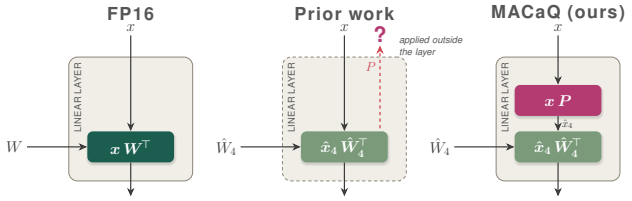

Figure 1. Prior work propagates the activation-side change of basis (labeled P) outside the linear layer to avoid runtime overhead. This incurs an engineering and quantization-time overhead, where for each layer a suitable absorption target must be found. MACaQ instead applies a lightweight transform online, preserving complete layer locality at negligible (ca. 2%) run-time overhead for sufficiently large layers.

In this paper, we argue for tolerating the overhead introduced by online transforms because keeping them online

provides a different benefit: *simplicity through layer locality*. MACaQ confines every transform and quantization decision to a single linear layer. No neighboring layer is modified to compensate for its quantization, and no basis transform needs to be propagated through the network. Eliminating these inter-layer dependencies allows each linear layer to be calibrated, transformed, and quantized independently. This simplifies application to new architectures, enables layers to be loaded and quantized one at a time, and leads to substantially faster quantization. Furthermore, *all* linear layers can benefit from online basis transforms, where in some architectures not all layers can have absorbed basis transforms.

The cost of this design choice is small. MACaQ uses only a per-channel scale and a block-Hadamard transform online. Their cost grows with the input dimension, whereas the cost of the subsequent matrix multiplication also grows with the output dimension. Consequently, their relative overhead diminishes for larger layers. We measure an end-to-end decode overhead of 2.3% for Qwen3-32B in vLLM, while obtaining a better quality-quantization-time trade-off than prior MXFP4 methods.

The main contributions of this paper are:

- We propose **MACaQ**, a strictly per-linear-layer pipeline for W4A4 MXFP4 quantization. MACaQ achieves higher output quality than state-of-the-art MXFP4 methods while requiring up to 11× less quantization time, see Fig. 2.
- We show that retaining lightweight basis transforms *online* incurs only a small inference overhead while **eliminating cross-layer dependencies**. This simplifies engineering, see Fig. 1, enables layer-at-a-time quantization with low memory requirements, and allows the same method to work on widely different model architectures.
- We introduce **Flip-Refinement**, a fast Hessian-aware coordinate-descent method that improves an initial round-to-nearest quantization assignment by moving individual FP4 codes to adjacent representable values.
- We provide efficient **Triton kernels** for MACaQ-quantized MXFP4 models in vLLM, achieving up to 3× end-to-end speedup over FP16 while limiting the overhead of online transforms to 2.3% on Qwen3-32B.

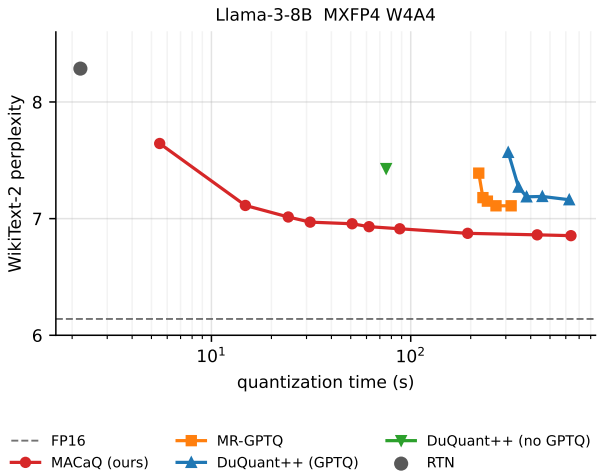

Figure 2. Quantization time vs. WikiText-2 perplexity on Llama-3-8B (OCP MXFP4 W4A4), all measured on the same hardware. MACaQ is run with different number of flips and calibration tokens, the other methods where possible scale by calibration tokens. MACaQ matches the best quality of these strong baselines in 11× less time.

Method

Problem Formulation

We consider weight and activation quantization to MXFP4, with the matrix multiplications themselves executed in MXFP4. An MXFP4 tensor is divided into blocks of $B = 32$ elements. Each block shares an E8M0 scale, represented by eight exponent bits and no mantissa bits, while each element has an individual four-bit FP4 E2M1 code consisting of a sign bit, two exponent bits, and one mantissa bit (25).

Consider a linear layer with weight matrix $W \in \mathbb{R}^{m\times n}$ and a calibration set of layer inputs $X \in \mathbb{R}^{N\times n}$. Similar to GPTQ (9), we seek to minimize the layer's output reconstruction error:

$$\min_{\widehat{W}} \left\| XW^\top - X\widehat{W}^\top \right\|_F^2 = \sum_{i=1}^{m} r_i H r_i^\top, \qquad (1)$$

where

$$H = X^\top X, \qquad r_i = w_i - \widehat{w}_i. \qquad (2)$$

Here, w_i is the i-th row of W, $\widehat{w}_i$ is its quantized reconstruction, and the input Hessian H is shared across all output rows.

MACaQ applies a per-channel scale followed by a block-Hadamard rotation. Let $D = \mathrm{Diag}(t)$ contain the per-channel scales and let B denote a block-diagonal Hadamard transform with blocks of size 32. We trans-

form the activations and weights as

$$X' = XDB, \qquad W' = WD^{-1}B. \tag{3}$$

Because B is orthogonal, the transformation preserves the unquantized linear operation:

$$X'W'^{\top} = XDB\left(WD^{-1}B\right)^{\top} = XW^{\top}. \tag{4}$$

The corresponding Hessian in the transformed basis is

$$H' = X'^{\top}X' = B^{\top}DHDB. \tag{5}$$

For the exponent selection and Flip-Refinement steps below, we use a ridge-regularized version

$$H_f = H' + \lambda \ \mathrm{mean}\big(\mathrm{diag}(H')\big)\, I. \tag{6}$$

We use $\lambda = 1.0$ by default.

At inference time, a MACaQ linear layer computes

$$y = [(x \odot t)B]_{\mathrm{MXFP4}} \left[\widehat{W}'^{\top}\right]_{\mathrm{MXFP4}}, \tag{7}$$

where $\widehat{W}'$ denotes the MXFP4 reconstruction of W' and $[\cdot]_{\mathrm{MXFP4}}$ denotes quantization to MXFP4. Thus, all activation-side transformations remain local to the linear layer.

MACaQ quantizes each layer in four steps: **(i)** per-channel balancing, **(ii)** block-Hadamard rotation, **(iii)** Hessian-weighted block exponent selection, and **(iv)** Flip-Refinement of the FP4 codes.

Online Scaling

The first transform is a per-channel scale t. It divides the weight columns before quantization and multiplies the corresponding activation channels:

$$xW^{\top} = (x \odot t)(W \oslash t)^{\top}. \tag{8}$$

The two operations cancel in the unquantized matrix multiplication, but change the distributions presented to the weight and activation quantizers.

The aim of this transformation is to reduce the magnitude of outlying activation channels while simultaneously balancing the weight columns. There is a natural trade-off: scaling down a given activation channel scales up the corresponding weight column, and vice versa (29). We choose

$$t_j = \frac{(\sigma_j^W)^{\alpha}}{(\mathrm{rms}_j^x)^{1-\alpha}}, \qquad \mathrm{rms}_j^x = \sqrt{\frac{H_{jj}}{N}}, \tag{9}$$

where σ_j^W is the standard deviation of weight column j and rms_j^x is the RMS magnitude of activation channel j, obtained directly from the diagonal of H. We use $\alpha = 0.75$, chosen on a validation set (a subset of wikitext-2 train).

Channels with large activation RMS receive smaller values of t_j and are therefore attenuated before activation quantization. Conversely, weight columns with large standard deviation receive larger values of t_j and are reduced by the division $W \oslash t$. The parameter α controls the trade-off between activation and weight balancing. We clamp the resulting scales to a finite range for numerical stability.

Online Micro-Rotation

After scaling, we apply a block-Hadamard rotation along the input-channel dimension. For the block-diagonal orthogonal matrix B,

$$xW^{\top} = (xB)(WB)^{\top}. \tag{10}$$

As in prior work (4), the Hadamard transform suppresses outliers by distributing concentrated channel energy across approximately Gaussian-distributed transformed channels.

We restrict the transform to blocks of size 32, matching the MXFP4 microscaling block size. This avoids spreading an outlying channel across multiple independently scaled MXFP4 blocks (17). The same fixed transform is applied to the scaled activations online and absorbed into the transformed weights offline.

Combining scaling and rotation gives

$$x' = (x \odot t)B, \qquad W' = (W \oslash t)B. \tag{11}$$

MACaQ first applies the per-channel scale and then the block-Hadamard transform, because the importance weighting of channels must happen in the unrotated domain.

Hessian-Weighted Exponent Selection

In MXFP4, every block shares a power-of-two E8M0 scale. Selecting this scale presents a trade-off between resolution and clipping: a smaller scale gives finer resolution around zero but may clip large values, whereas a larger scale avoids clipping at the cost of coarser quantization.

Before refining the individual FP4 codes, MACaQ screens a small set of candidate exponents independently for every weight row and MXFP4 block. Let $\mathcal{B}$ denote one

block of transformed weight row i. For each candidate exponent e, we round the block to the corresponding E2M1 grid and compute

$$E_{i,\mathcal{B}}(e) = \sum_{j \in \mathcal{B}} (H_f)_{jj} \left(W'_{ij} - \widehat{W}'_{ij}(e) \right)^2. \tag{12}$$

We select the exponent that minimizes $E_{i,\mathcal{B}}(e)$.

We evaluate offsets $\{+1, 0, -1\}$ around the standard exponent derived from the block maximum. The selected block exponent is then held fixed during Flip-Refinement.

Flip-Refinement

The exponent screen produces an initial round-to-nearest assignment of the FP4 codes. Starting from this assignment, Flip-Refinement minimizes the ridge-regularized Hessian objective by moving individual codes to adjacent E2M1 levels.

For output row i, let

$$r_i = w'_i - \widehat{w}'_i, \qquad g_i = H_f r_i^\top. \tag{13}$$

Suppose changing code (i, j) changes its reconstructed value by Δ, such that

$$\widehat{W}'_{ij} \leftarrow \widehat{W}'_{ij} + \Delta. \tag{14}$$

The residual then changes from r_i to $r_i - \Delta e_j$. The resulting decrease in the row objective is

$$\text{gain}(i, j, \Delta) = 2\Delta g_{i,j} - \Delta^2 (H_f)_{jj}. \tag{15}$$

A positive gain implies an improving move. After accepting the flip, the gradient is updated incrementally:

$$g_i \leftarrow g_i - \Delta (H_f)_{:,j}. \tag{16}$$

This update avoids recomputing the complete matrix-vector product after each flip.

The E2M1 grid is non-uniform, so Δ depends on the current code, the chosen direction, and the fixed block scale. At every iteration, we evaluate the adjacent levels of every code and select the single highest-gain move for each output row. The output rows are independent and are processed in parallel. Rows for which no positive-gain move remains are removed from the active set and require no further work. Algorithm 1 summarizes the procedure.

Here, $\Delta \in \mathcal{N}(Q_{ij}; s_{ij})$ are the reconstructed-value changes associated with moving one level upward or downward on the local FP4 grid (where possible).

Algorithm 1: Flip-Refinement for one linear layer

Require: transformed weights $W' \in \mathbb{R}^{m \times n}$; regularized Hessian H_f; initial FP4 codes Q; fixed block scales s; maximum iterations K
Ensure: refined FP4 codes Q

for $i \leftarrow 1$ **to** m **in parallel do**
  $\widehat{w}'_i \leftarrow dq(Q_i; s_i)$
  $r_i \leftarrow w'_i - \widehat{w}'_i$
  $g_i \leftarrow H_f r_i^\top$
end for
$\mathcal{A} \leftarrow \{1, \ldots, m\}$
for $k \leftarrow 1$ **to** K **do**
  for $i \in \mathcal{A}$ **in parallel do**
    $(j_i, \Delta_i) \leftarrow \arg\max_{j,\Delta} \left[2\Delta g_{i,j} - \Delta^2 (H_f)_{jj} \right]$
    $\gamma_i \leftarrow 2\Delta_i g_{i,j_i} - \Delta_i^2 (H_f)_{j_i j_i}$
  end for
  $\mathcal{I} \leftarrow \{i \in \mathcal{A} : \gamma_i > 0\}$
  if $\mathcal{I} = \emptyset$ **then**
    break
  end if
  for $i \in \mathcal{I}$ **in parallel do**
    $Q_{i,j_i} \leftarrow$ selected adjacent code
    $g_i \leftarrow g_i - \Delta_i (H_f)_{:,j_i}$
  end for
  $\mathcal{A} \leftarrow \mathcal{I}$
end for
return Q

Each iteration accepts at most one flip per output row, so K is also the maximum number of accepted flips per row. In practice, many rows converge before reaching this budget.

Cost of Online Transforms

The activation-side pipeline of a MACaQ linear layer is: **(i)** apply the per-channel scale, **(ii)** apply the block-Hadamard transform, and **(iii)** quantize the result to MXFP4. Activation quantization is required by any W4A4 MXFP4 implementation, so the additional operations introduced by MACaQ are only the scale and the micro-rotation.

For a layer processing T tokens with input dimension n and output dimension m, a block-Hadamard transform of size B requires approximately $Tn \log_2 B$ additions, while the scale requires Tn multiplications. By comparison, the matrix multiplication performs approximately $2Tmn$

arithmetic operations. The ratio is therefore

$$\frac{C_{\text{online}}}{C_{\text{matmul}}} = \frac{Tn(\log_2 B + 1)}{2Tmn} = \frac{\log_2 B + 1}{2m}. \tag{17}$$

For $B = 32$, this becomes

$$\frac{C_{\text{online}}}{C_{\text{matmul}}} = \frac{3}{m}. \tag{18}$$

Both online transforms are applied once to each input and reused across all m output channels. Their cost is therefore independent of m, whereas the cost of the matrix multiplication grows linearly with m. For example, the smallest output dimension among the quantized linear layers of Llama-3-8B is $m = 1024$, giving an arithmetic-operation ratio below 0.3%.

This estimate describes arithmetic work rather than wall-clock performance. The measured overhead also depends on kernel fusion, memory traffic, launch overhead, and the relative throughput of FP16 and MXFP4 operations. We therefore measure the actual kernel-level and end-to-end overhead in later sections. Consistent with the analysis above, the relative overhead decreases with layer and model size.

Quantization Procedure

MACaQ preserves the layer-wise modularity of the underlying network: each linear layer has its own scaling factors, block rotation, Hessian, block exponents, and FP4 codes. Quantizing one linear layer never requires modifying a neighboring normalization layer, projection, or residual branch.

Forward hooks accumulate

$$H_\ell = \sum_k X_{\ell,k}^\top X_{\ell,k} \tag{19}$$

for every linear layer ℓ over chunks of the calibration sequence. After collecting these statistics, each linear layer is independently scaled, rotated, exponent-screened, and refined. The original layer is then replaced by its packed MXFP4 implementation.

Only the layer currently being processed needs to reside on the GPU (and for the calibration, outputs of parallel layers). The remaining model weights remain in host memory. This streaming procedure reduces peak device memory and allows the same quantization implementation to be reused across model architectures; all calibration and quantization logic is attached locally to linear layers.

Implementation Details

MACaQ offers two levers for trading quantization time against output quality: the number of calibration tokens and the maximum number K of Flip-Refinement iterations.

We produce the Pareto curve in Fig. 2 by sweeping these two parameters. Quality begins to plateau at approximately 2×10^6 calibration tokens and $K = 512$ Flip-Refinement iterations. For fair comparison we use $K = 128$ and 2×10^5 calibration tokens for MACaQ in the experimental section (ca. 150s to quantize Llama3-8B).

Besides this, our default configuration uses block size 32, exponent offsets $\{+1, 0, -1\}$, scale-balancing parameter $\alpha = 0.75$, and ridge coefficient $\lambda = 1.0$. The parameters α and λ were chosen with a wikitext train set validation run on Llama3-8B (sensitivity plot in appendix). The Flip-Refinement implementation parallelizes across output rows and removes converged rows from the active set. We additionally compile the repeated refinement step with `torch.compile`; the initial compilation cost is amortized across layers with matching shapes.

Experiments

To show the effectiveness of MACaQ, we consider several different metrics. For comparison with literature baselines, we use model perplexity, as it is widely reported in quantization work and provides a continuous and comparably reliable measure of model quality.

We further report results on generative benchmarks. In generative settings, quantization errors may accumulate over generated sequences in ways that are not visible in fixed-text metrics such as perplexity, and these tasks are closer to many practical use cases. To illustrate the ease of generalization to new architectures, we also apply MACaQ to an image diffusion model. Finally, we measure the throughput of MACaQ relative to standard FP16 and other common baselines, both at the kernel level and end-to-end, and perform an ablation over the main components of MACaQ.

The models we evaluate are drawn from the Llama-3 (11), Qwen3 (30), and Qwen3.5 (23) families. We evaluate perplexity on Llama models for comparability with prior work and use the reasoning-capable Qwen models for generative reasoning evaluations. For image diffusion, we use FLUX.2 Klein 4B (3), as it is openly available.

Note that for MoE models we only quantize expert layers (also for baselines) otherwise we quantize all weights

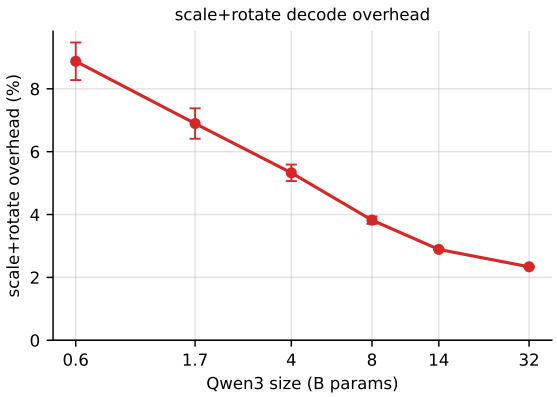

Figure 3. Decode overhead introduced by online rotation and scaling (batch 1, context 512). The overhead decreases with model size and reaches approximately 2.3% for Qwen3-32B. Error bars show the standard error.

and activations, but not the embeddings and lm-head (as is customary).

All experiments are run on a recent GPU with 504 TFLOPS (FP16), 96 GB memory and 1.79 TB/s memory bandwidth and use OCP MXFP4 format.

Perplexity

For perplexity measurements, we follow related work and calibrate on the WikiText-2 training set. We evaluate on the WikiText-2 (20) and C4 (24) test sets. WikiText-2 can therefore be understood as an in-distribution evaluation with respect to the calibration data, while C4 provides an out-of-distribution evaluation. We use a context length of 2048. The results in Tab. 1 and Fig. 2 show that MACaQ matches or outperforms the baselines also in terms of quality, not only speed.

Large-Model Quantization

MACaQ can be used to efficiently quantize models that are too large to reside on the target GPU at full precision. To demonstrate this, we quantize Qwen3.5-122B-A10B on a machine with a single 96GB GPU. The model contains 122B total parameters, and its FP16 weights require ca. 244GB.

MACaQ processes the model with lazy loading of layers and reaches a GPU-memory usage of 32.5GB. Quantization takes 64 minutes (with 32 flips and 512K calibration tokens) and produces a WikiText-2 perplexity of 5.14, compared with 5.06 for the full-precision model. The resulting MXFP4 model requires 74GB of GPU memory and can run on the same 96GB GPU. Thus, MACaQ can quantize a model that does not fit on the target GPU in its original precision into one that does.

Generative Language Tasks

Generative benchmarks are closer to many practical workloads and have the potential to reveal accumulation effects that may only appear when a model generates a sequence of tokens. They also avoid the limitations of multiple-choice QA evaluation highlighted by Dutta et al. (5).

We evaluate MACaQ-quantized dense Qwen3 and Qwen3.5 MoE models on IFEval (31) and MATH-500 (13). MATH-500 contains free-form, non-multiple-choice mathematics problems that require reasoning chains to answer correctly. IFEval tests the instruction following capabilities of models in generation.

Tab. 2 shows that MACaQ performs well on these tasks.

Image Diffusion

One of the key distinguishing features of MACaQ is that it can be applied to new architectures with minimal effort because it treats each linear layer in isolation. We demonstrate this by quantizing the linear layers of FLUX.2 Klein 4B (3) to MXFP4.

This model also offers an official NVFP4 variant[1], which provides a strong baseline. NVFP4 uses microscaling groups of 16, rather than the 32 used by MXFP4, and requires approximately 4.5 rather than 4.25 bits per element.

We find that MACaQ produces a more faithful quantized model with respect to the original BF16 model when prompted with GenEval2 prompts (14). In Table 3, we report pairwise metrics ranging from signal-level similarity (PSNR), through perceptual similarity (LPIPS), to semantic similarity (DINOv3). MACaQ is closer to the original BF16 model across all these metrics. For a qualitative illustration see Fig. 4.

Kernel-Level Speed

In Table 4, we compare the speed of a single linear layer across different batch sizes and matrix dimensions. We compute $\mathbf{X}\mathbf{W}^\top$ using square weight matrices $\mathbf{W} \in \mathbb{R}^{N\times N}$, where $N \in \{4096, 16384\}$, and inputs $\mathbf{X} \in \mathbb{R}^{B\times L\times N}$, where the batch size $B \in \{1, 16\}$ and context length

[1] for a fair comparison we quantize the same layers that this official NVFP4 model quantizes with MACaQ

Table 1. OCP MXFP4 W4A4, WikiText2 / C4 perplexity (↓). Best quantized result per column in bold, second best underlined.

	Llama-3-8B		Llama-3.2-3B		Llama-3-8B-Inst.		Llama-3.1-8B-Inst.		Qwen3-32B		Qwen3.5-35B-A3B	
Method	Wiki2	C4	Wiki2	C4	Wiki2	C4	Wiki2	C4	Wiki2	C4	Wiki2	C4
FP16	6.14	9.46	7.81	11.34	8.31	13.03	7.16	11.41	7.62	12.45	6.55	10.98
QuaRot (RTN)	9.34	15.53	12.40	18.63	11.74	19.21	10.00	16.48	10.19	16.59	6.77	11.38
QuaRot (GPTQ)	7.65	12.62	9.65	15.11	9.62	15.85	8.58	14.49	8.46	13.42	6.79	11.33
FlatQuant	7.21	11.65	9.00	14.87	9.25	15.27	<u>8.10</u>	13.40	8.11	13.28	6.87	11.57
MR-GPTQ	<u>7.13</u>	<u>11.37</u>	<u>8.92</u>	<u>13.51</u>	9.13	<u>14.62</u>	8.14	<u>13.24</u>	8.15	13.27	6.75	<u>11.22</u>
DuQuant++	7.48	11.84	9.36	14.19	9.37	14.86	8.41	13.67	8.27	13.33	<u>6.73</u>	11.28
DuQuant++ (GPTQ)	7.17	11.66	9.02	13.92	<u>9.04</u>	14.64	8.18	13.48	<u>8.07</u>	<u>13.15</u>	6.75	11.26
MACaQ (ours)	**6.90**	**11.01**	**8.61**	**13.07**	**8.90**	**14.41**	**7.88**	**12.91**	**7.94**	**12.93**	**6.66**	**11.21**

Table 2. OCP MXFP4 W4A4. **IFEval** strict prompt-level accuracy (↑, average over k=10; ± across-seed standard error). **MATH500** accuracy (↑, average over k=10; ± across-seed standard error). Best quantized result per column in bold, second best underlined; also all results within one standard error of the baseline are bolded.

	Qwen3-32B		Qwen3.5-35B-A3B	
Method	IFEval	MATH500	IFEval	MATH500
Original (FP16)	83.60%±0.25	97.78%±0.10	85.29%±0.25	99.38%±0.06
QuaRot (RTN)	78.46%±0.23	93.65%±0.20	**85.41%**±0.33	98.78%±0.10
QuaRot (GPTQ)	82.35%±0.29	96.98%±0.19	**85.60%**±0.39	98.60%±0.10
MR-GPTQ	<u>82.81%</u>±0.27	**97.44%**±0.16	**85.43%**±0.24	<u>98.86%</u>±0.07
DuQuant++ (RTN)	81.57%±0.29	96.82%±0.21	85.29%±0.20	98.58%±0.11
DuQuant++ (GPTQ)	<u>82.81%</u>±0.27	96.90%±0.14	**85.59%**±0.21	98.50%±0.11
MACaQ (ours)	**83.20%**±0.26	<u>97.26%</u>±0.11	**85.42%**±0.40	**98.90%**±0.09

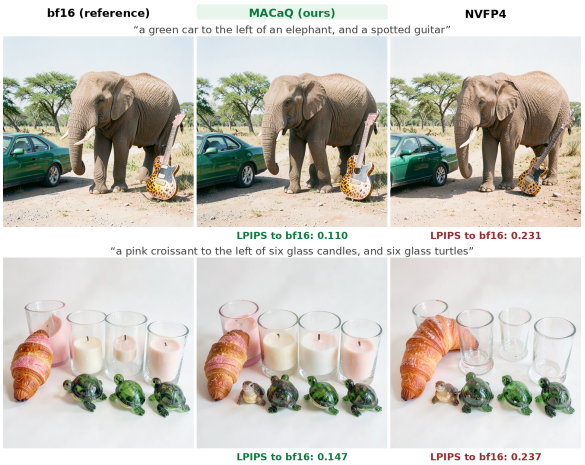

Figure 4. Comparison of FLUX2-klein-4B under MACaQ MXFP4 vs. the official NVFP4 quantization. MACaQ is more similar to the high precision model; note e.g. the shadow's orientation in the first image and the empty glasses in the second. Quantitative results in Tab. 3.

$L \in \{512, 16384\}$. We compare our MACaQ MXFP4 linear layer with, first, the same kernel with online scaling and rotation disabled; second, the Marlin W4A16 kernel (10); and, finally, a FP16 baseline.

The MACaQ kernel reaches a substantial speedup over FP16, while the overhead of online scaling and rotation remains low. For the smaller layer, the overhead is at most approximately 6%, and for the larger layer it remains below 3%. As expected from the theoretical analysis, the relative overhead decreases for larger layers.

End-to-End Speed

On Qwen3-32B at batch 1, our MACaQ MXFP4 implementation decodes at 69.8 tok/s against 22.8 for FP16 (3.06x); at batch 256 the gain is 5252.8 vs 3061.5 tok/s. Per-batch-size numbers for both models are in the appendix. MACaQ achieves up to 3× end-to-end speed-up over FP16. Generally, speed-ups are comparable to Marlin (10) in the memory-bound case and outperform Marlin

Table 3. FLUX.2-klein-4B: MACaQ vs. the official NVFP4 QAD checkpoint. GenEval2 (800 prompts), 1024^2, 6 steps, DiT-only W4A4. PSNR/SSIM/LPIPS and DINOv3 (ViT-L/16) feature cosine are paired against the BF16 output (same prompt and seed). Peak VRAM at batch 1. MACaQ is closer to BF16 on all four fidelity metrics and ties on VRAM.

Method	Format	bits/w	PSNR ↑	SSIM ↑	LPIPS ↓	$\text{DINO}_{\text{global}}$ ↑	$\text{DINO}_{\text{patch}}$ ↑	VRAM (GB) ↓
Full Precision	BF16	16.0	-	-	-	-	-	18.5
MACaQ (ours)	MXFP4	4.25	**15.04**	**0.697**	**0.218**	**0.882**	**0.947**	**13.1**
Official NVFP4	NVFP4	4.50	14.85	0.676	0.235	0.878	0.937	13.3

Table 4. Kernel-level speed of one linear layer $\mathbf{X} \cdot \mathbf{W}^\top$, $\mathbf{W} \in \mathbb{R}^{N \times N}$, $\mathbf{X} \in \mathbb{R}^{B \times L \times N}$ ($M = B \cdot L$ rows; $L = 1$ is decode). Median as **speedup relative to fp16** (↑; fp16≡ 1.00). Standard Error is omitted as it is below 0.02 in all cases. MACaQ is the MXFP4 W4A4 kernel with online per-channel scaling and block-Hadamard rotation; −SR disables both; Marlin is the W4A16 kernel of Frantar et al. (10). "SR cost" is the relative slowdown from enabling scale+rotation.

N	B	L	MACaQ	−SR	Marlin	FP16	SR cost
4096	1	1	**1.31**	1.31	**1.31**	1.00	<1%
	1	16384	**1.75**	1.86	1.02	1.00	6.4%
	16	1	**1.75**	1.75	**1.75**	1.00	<1%
	16	16384	**1.50**	1.57	1.01	1.00	4.1%
16384	1	1	5.04	5.11	**8.26**	1.00	1.3%
	1	16384	**2.01**	2.05	1.00	1.00	1.6%
	16	1	4.47	4.55	**6.19**	1.00	1.7%
	16	16384	**1.83**	1.85	0.98	1.00	0.9%

in compute-bound settings.

In Fig. 3, we isolate the overhead caused by online rotation and scaling. For the small Qwen3-0.6B model, this overhead is approximately 8%(i.e., MACaQ still provides a substantial speedup over FP16). As explained in the Method section, the overhead decreases with model size, reaching approximately 2% for Qwen3-32B.

Ablation

In Table 5, we show the contributions of the main components of MACaQ. Each component provides an improvement of a similar order of magnitude, and their combination achieves the lowest quantization error.

Table 5. Additive contribution of each MACaQ component on Llama-3-8B (MXFP4 W4A4), WikiText-2 perplexity at 512 Flips and 200'000 calibration tokens.

Configuration	Wiki2 ↓	Δ
Full Precision (FP16)	6.14	–
RTN	8.28	–
+ Hessian exponent search	8.09	−0.19
+ micro-rotation	7.65	−0.44
+ online second scale	7.08	−0.57
+ flip refinement (= MACaQ)	6.87	−0.21

Related Work

Hessian-Based Weight Compression Much of the current *second-order* quantization literature builds on Optimal Brain Damage (OBD) (15) and Optimal Brain Surgeon (OBS) (12) that introduced Hessian-based model compression. Optimal Brain Compression (OBC) (8) adapted this framework to layer-wise compression and introduced Optimal Brain Quantization (OBQ), using the layer-input Hessian $H = X^\top X$ to minimize output reconstruction error. GPTQ (9) subsequently made this approach scalable to large language models by fixing the quantization order, allowing the same Hessian factorization to be reused across output rows. These methods assign weights sequentially and compensate the remaining unquantized weights. In contrast, MACaQ begins from a cheap RTN assignment and then revisits the individual codes through incremental Hessian-aware updates.

Coordinate-Descent Quantization QuantEase (2) and CDQuant (22) also formulate quantization (pure weight-quantization) as coordinate descent and improve over one-shot assignments such as GPTQ by revisiting quantized weights. Our Flip-Refinement is designed primarily for speed in weight and activation quantization. It keeps each MXFP4 block exponent fixed and considers only the two E2M1 levels adjacent to the current code, allowing closed-form evaluation of the gain of each move.

Basis Transforms Basis transforms have become a central tool for reducing weight and activation quantiza-

tion error. QuIP (4) popularized incoherence processing through Hadamard transforms for weight quantization, and QuaRot (1) extended Hadamard rotations to weight–activation quantization. SmoothQuant (29) used element-wise scaling to balance errors between activations and weights. Later work has optimized the rotation itself (19, 16), general per-layer affine maps, as in FlatQuant (28), or an element-wise scale, including weight-only quantization with AWQ (18) and calibration-free weight quantization with SINQ (21)

For microscaling formats, block-local rotations are often preferable to full-channel rotations, e.g. MR-GPTQ (6) and (27, 17), as they avoid spreading an outlier across multiple independently scaled quantization blocks. Other work has studied how these transforms interact: (7) analyze the sources of error in weight–activation quantization and recommend combining rotation and scaling, while (26) show that reducing concentrated activation mass makes Hadamard transforms more effective preconditioners.

Limitations

The core design decision of MACaQ, keeping everything layer-local, means that cross-layer error compensation is impossible. A secondary pass with cross-layer statistics could be added on top, but would likely incur the slower quantization times of other methods.

In our MoE experiments we restricted ourselves to expert quantization only due to practicality considerations.

For the smallest model tested (0.6B parameters) the online basis transform overhead climbs to 8%, which starts to be relevant for some settings.

Conclusion

We have introduced MACaQ, a fast and easy to apply MXFP4 quantization method for models based on linear layers. MACaQ matches or outperforms state of the art methods in quality and is faster. The key concepts that enable this are a light-weight Flip-Refinement to replace GPTQ, a novel Hessian-aware shared-scale optimization, and light-weight *online* basis transforms that enable the quantization processing to be local to each linear layer. All these features combined enable us to quantize a 122B Qwen3.5 model in 64 minutes while using only 32.5GB GPU RAM.

References

[1] Saleh Ashkboos, Amirkeivan Mohtashami, Maximilian L Croci, Bo Li, Pashmina Cameron, Martin Jaggi, Dan Alistarh, Torsten Hoefler, and James Hensman. Quarot: Outlier-free 4-bit inference in rotated llms. *Advances in Neural Information Processing Systems*, 37:100213–100240, 2024.

[2] Kayhan Behdin, Ayan Acharya, Aman Gupta, Qingquan Song, Siyu Zhu, Sathiya Keerthi, and Rahul Mazumder. Quantease: Optimization-based quantization for language models, 2023. URL https://arxiv.org/abs/2309.01885.

[3] The Black Forest Labs. FLUX.2: Frontier Visual Intelligence. https://bfl.ai/blog/flux-2, 2025.

[4] Jerry Chee, Yaohui Cai, Volodymyr Kuleshov, and Christopher De Sa. Quip: 2-bit quantization of large language models with guarantees. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 4396–4429. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/0df38cd13520747e1e64e5b123a78ef8-Paper-Conference.pdf.

[5] Abhinav Dutta, Sanjeev Krishnan, Nipun Kwatra, and Ramachandran Ramjee. Accuracy is not all you need. *Advances in Neural Information Processing Systems*, 37:124347–124390, 2024.

[6] Vage Egiazarian, Roberto L. Castro, Denis Kuznedelev, Andrei Panferov, Eldar Kurtic, Shubhra Pandit, Alexandre Noll Marques, Mark Kurtz, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. Bridging the gap between promise and performance for microscaling FP4 quantization. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=zCBGe9AqJZ.

[7] Marco Federici, Boris van Breugel, Paul N. Whatmough, and Markus Nagel. Dissecting quantization error: A concentration-alignment perspective. In *Forty-third International Conference on Machine Learning*, 2026. URL https://openreview.net/forum?id=7BoFyTPqMr.

[8] Elias Frantar and Dan Alistarh. Optimal brain compression: A framework for accurate post-training quantization and pruning. *Advances in Neural Information Processing Systems*, 35:4475–4488, 2022.

[9] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.

[10] Elias Frantar, Roberto L Castro, Jiale Chen, Torsten Hoefler, and Dan Alistarh. Marlin: Mixed-precision auto-regressive parallel inference on large language models. In *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, pages 239–251, 2025.

[11] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[12] Babak Hassibi, David Stork, and Gregory Wolff. Optimal brain surgeon: Extensions and performance comparisons. In J. Cowan, G. Tesauro, and J. Alspector, editors, *Advances in Neural Information Processing Systems*, volume 6. Morgan-Kaufmann, 1993. URL https://proceedings.neurips.cc/paper_files/paper/1993/file/b056eb1587586b71e2da9acfe4fbd19e-Paper.pdf.

[13] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *NeurIPS*, 2021.

[14] Amita Kamath, Kai-Wei Chang, Ranjay Krishna, Luke Zettlemoyer, Yushi Hu, and Marjan Ghazvininejad. Geneval 2: Addressing benchmark drift in text-to-image evaluation. *arXiv preprint arXiv:2512.16853*, 2025.

[15] Yann LeCun, John Denker, and Sara Solla. Optimal brain damage. *Advances in neural information processing systems*, 2, 1989.

[16] Haokun Lin, Haobo Xu, Yichen Wu, Jingzhi Cui, Yingtao Zhang, Linzhan Mou, Linqi Song, Zhenan Sun, and Ying Wei. Duquant: Distributing outliers via dual transformation makes stronger quantized llms. *Advances in Neural Information Processing Systems*, 37:87766–87800, 2024.

[17] Haokun Lin, Xinle Jia, Haobo Xu, Bingchen Yao, Xianglong Guo, Yichen Wu, Zhichao Lu, Ying Wei, Qingfu Zhang, and Zhenan Sun. Duquant++: Fine-grained rotation enhances microscaling fp4 quantization. *arXiv preprint arXiv:2604.17789*, 2026.

[18] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. Awq: Activation-aware weight quantization for on-device llm compression and acceleration. *Proceedings of machine learning and systems*, 6:87–100, 2024.

[19] Zechun Liu, Changsheng Zhao, Igor Fedorov, Bilge Soran, Dhruv Choudhary, Raghuraman Krishnamoorthi, Vikas Chandra, Yuandong Tian, and Tijmen Blankevoort. Spinquant: Llm quantization with learned rotations. In *The Thirteenth International Conference on Learning Representations*, 2025.

[20] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models, 2016.

[21] Lorenz K Müller, Philippe Bich, Jiawei Zhuang, Ahmet Çelik, Luca Benfenati, and Lukas Cavigelli. Sinq: Sinkhorn-normalized quantization for calibration-free low-precision llm weights. *arXiv preprint arXiv:2509.22944*, 2025.

[22] Pranav Ajit Nair and Arun Sai Suggala. Cdquant: Greedy coordinate descent for accurate llm quantization, 2024. URL https://arxiv.org/abs/2406.17542.

[23] The Qwen Team. Qwen3. 5-omni technical report. *arXiv preprint arXiv:2604.15804*, 2026.

[24] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21(140): 1–67, 2020.

[25] Bita Darvish Rouhani, Ritchie Zhao, Ankit More, Mathew Hall, Alireza Khodamoradi, Summer Deng, Dhruv Choudhary, Marius Cornea, Eric Dellinger, Kristof Denolf, et al. Microscaling data formats for deep learning. *arXiv preprint arXiv:2310.10537*, 2023.

[26] Sai Sanjeet, Ian Colbert, Pablo Monteagudo-Lago, Giuseppe Franco, Yaman Umuroglu, and Nicholas J. Fraser. Pushing the limits of block rotations in post-training quantization. In *Forty-third International Conference on Machine Learning*, 2026. URL https://openreview.net/forum?id=nvehxSdMqg.

[27] Yuantian Shao, Peisong Wang, Yuanteng Chen, Chang Xu, Zhihui Wei, and Jian Cheng. Block rotation is all you need for mxfp4 quantization. *arXiv preprint arXiv:2511.04214*, 2025.

[28] Yuxuan Sun, Ruikang Liu, Haoli Bai, Han Bao, Kang Zhao, Yuening Li, Xianzhi Yu, Lu Hou, Chun Yuan, Xin Jiang, et al. Flatquant: Flatness matters for llm quantization. In *Forty-second International Conference on Machine Learning*, 2025.

[29] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International conference on machine learning*, pages 38087–38099. PMLR, 2023.

[30] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[31] Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models, 2023. URL https://arxiv.org/abs/2311.07911.

Appendix

End to end speed table

Here we show detailed end to end speed results in Tab. 6.

Table 6. End-to-end throughput (median tok/s; 512-token prompt, vLLM graph mode) across batch sizes. Prefill=prompt/TTFT, decode=per-token rate, aggregated over the batch. MACaQ−SR disables the online scale+rotation (speed-only ablation).

	Llama-3-8B		Qwen3-32B	
Method	Prefill	Decode	Prefill	Decode
Batch 1				
bf16	38,171	93.2	11,027	22.8
Marlin W4A16	**71,760**	223.9	**29,307**	**71.8**
MACaQ (ours)	70,547	**227.9**	28,473	69.8
MACaQ −SR	72,852	232.9	28,364	69.4
Batch 16				
bf16	236,101	1383.9	57,621	342.7
Marlin W4A16	**442,456**	**3034.9**	**147,688**	**994.6**
MACaQ (ours)	436,001	2842.1	143,225	949.4
MACaQ −SR	467,704	3058.2	147,370	976.9
Batch 32				
bf16	393,524	2471.6	107,240	652.1
Marlin W4A16	599,408	**5251.5**	203,672	**1792.6**
MACaQ (ours)	**650,692**	5028.2	**236,880**	1756.7
MACaQ −SR	665,454	5426.4	243,237	1804.2
Batch 64				
bf16	607,508	4527.5	171,756	1234.7
Marlin W4A16	748,422	7750.3	226,696	2697.8
MACaQ (ours)	**904,177**	**7840.2**	**294,347**	**2709.8**
MACaQ −SR	954,273	8418.5	305,896	2836.7
Batch 128				
bf16	816,722	7566.8	224,958	2086.1
Marlin W4A16	915,282	9502.0	266,406	3170.0
MACaQ (ours)	**1,087,913**	**11158.3**	**323,884**	**3983.3**
MACaQ −SR	1,112,510	11345.3	333,024	4080.4
Batch 256				
bf16	965,564	10146.8	243,291	3061.5
Marlin W4A16	1,020,529	11220.3	276,373	3573.3
MACaQ (ours)	**1,157,387**	**14207.6**	**373,713**	**5252.8**
MACaQ −SR	1,143,268	14899.2	375,743	5402.9

Table 7. Quantization cost vs. quality on Llama-3-8B, MXFP4 W4A4. *Calib. Blocks* is the calibration budget in units of 2048 tokens: for the GPTQ-based baselines one block is exactly one 2048-token calibration sequence, while MACaQ consumes a flat token stream (its rungs are round decimal token counts, hence the fractional blocks). PPL is WikiText-2 perplexity; FP16 is 6.14. The * configuration uses only rotation and exponent screen.

Method	Flips	Calib. Blocks	Time (s)	PPL
FP16	–	–	–	6.14
RTN	–	0	2.2	8.29
MACaQ (ours)	0*	0	5.5	7.64
	0	1	14.8	7.11
	16	2	24.3	7.01
	32	4	31.3	6.97
	64	7.8	50.8	6.96
	64	16	61.8	6.93
	128	31.3	88.1	6.91
	256	125	193.1	6.87
	512	488.3	431.0	6.86
	512	976.6	636.9	6.85
MR-GPTQ	–	4	219.5	7.39
	–	16	230.3	7.18
	–	32	242.5	7.15
	–	64	267.9	7.11
	–	128	319.1	7.11
DuQuant++ (GPTQ)	–	4	308.5	7.57
	–	16	347.4	7.27
	–	32	381.3	7.19
	–	64	457.5	7.19
	–	128	624.4	7.16

Pareto-Plot Numerical Details

Here we report the numbers relating to the plot in Fig. 2 in Tab. 7.

Hyperparameter sensitivity

We find that MACaQ is not very sensitive to hyperparameter selection (see Fig. 5)

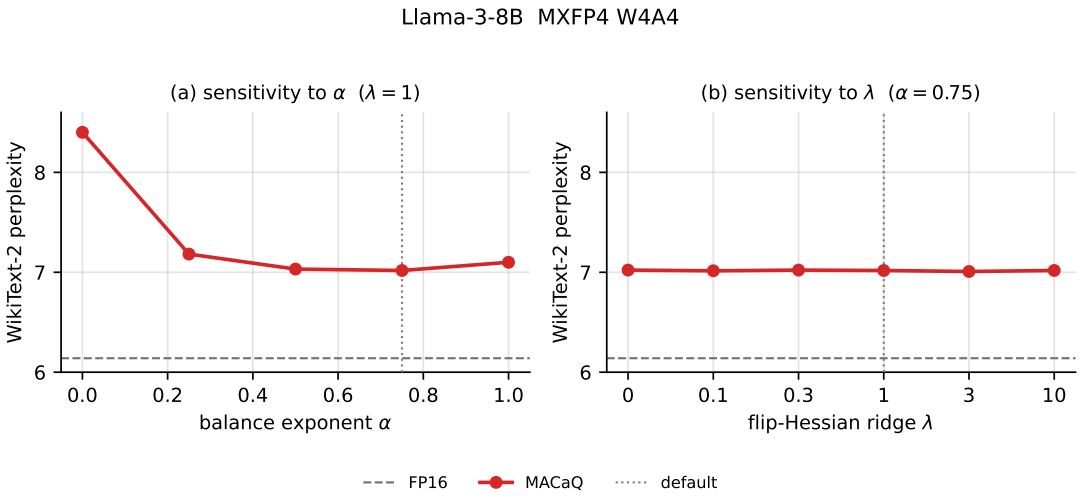

Figure 5. Hyperparameter sensitivity of MACaQ on Llama3-8B in the 4K token 16 flips configuration. Because the λ curve is flat, we choose a conservative value of 1.0 to prevent possible under-calibration issues.

OCP MXFP4 compliance

The OCP MXFP4 specification requires the shared block scale to be a power of two (E8M0). The DuQuant++ codebase provides both `cast_to_eBm0()` and `cast_to_eBm0_improved()`, the latter permitting fractional-power-of-two scales, which are more expressive but not representable in the E8M0 scale field. We use a compliant cast throughout, for all methods including our own, which accounts for a small discrepancy with some of the numbers reported in the DuQuant++ paper.

Additional Generated Images

Here we add further images generated by the FLUX2 baseline and quantized models for GenEval2[2] prompts and some simple photographic prompts, chosen to highlight the advantages of our model.

Extending Flip-Refinement to Vector-Quantized Weights

Flip-Refinement can also refine vector-quantized weights by replacing a codeword rather than a scalar code. We apply this extension to the `IQ3_S` and `IQ3_XXS` weight-only formats in `llama.cpp`.

Codebook constraints. Both formats encode four consecutive weight magnitudes with one codebook index: `IQ3_S` has 512 codewords and `IQ3_XXS` has 256. Each group of 32 weights shares a 4-bit scale code k, and each group of 256 weights shares an FP16 scale d. A reconstructed weight is $\widehat{w}_j = d\, f(k)\, \sigma_j c_j$, where c is a codeword and $\sigma_j \in \{\pm 1\}$. The scale factors are $f(k) = 1 + 2k$ for `IQ3_S` and $f(k) = (k + 0.5)/2$ for `IQ3_XXS`. Signs are independent in `llama.cpp`'s `IQ3_S` format; `IQ3_XXS` permits only the 128 even-parity sign patterns within each group of eight weights.

Changing one magnitude while holding the others fixed often produces a tuple outside the codebook. Scalar Flip-Refinement therefore has few legal moves, motivating a search over complete codewords.

Block refinement. We retain the regularized row objective from Sec. , $L_i = r_i H_f r_i^\top$ with $r_i = w'_i - \widehat{w}'_i$ and $g_i = H_f r_i^\top$. Recall that for a scalar move of size Δ on input j, Eq. (15) gives the exact decrease $2\Delta g_{i,j} - \Delta^2 (H_f)_{jj}$. A block move is the same statement with vectors. Let $\mathcal{B}$ contain the four input indices of one block, let v and u be its current and candidate reconstructions, and write $A = (H_f)_{\mathcal{B},\mathcal{B}}$ and $\delta = u - v$. Then

$$\text{gain}(i, \mathcal{B}, u) = 2\, \delta^\top g_{i,\mathcal{B}} - \delta^\top A\, \delta. \tag{20}$$

The expression is exact because the objective is quadratic: a change confined to four coordinates is described completely by the gradient on those coordinates and the 4×4 Hessian block. Interactions with the other $n - 4$ weights of the row are already contained in $g_{i,\mathcal{B}}$.

Eq. (20) has a geometric reading that makes the search

[2] from https://github.com/facebookresearch/GenEval2 under CC BY-NC 4.0 license

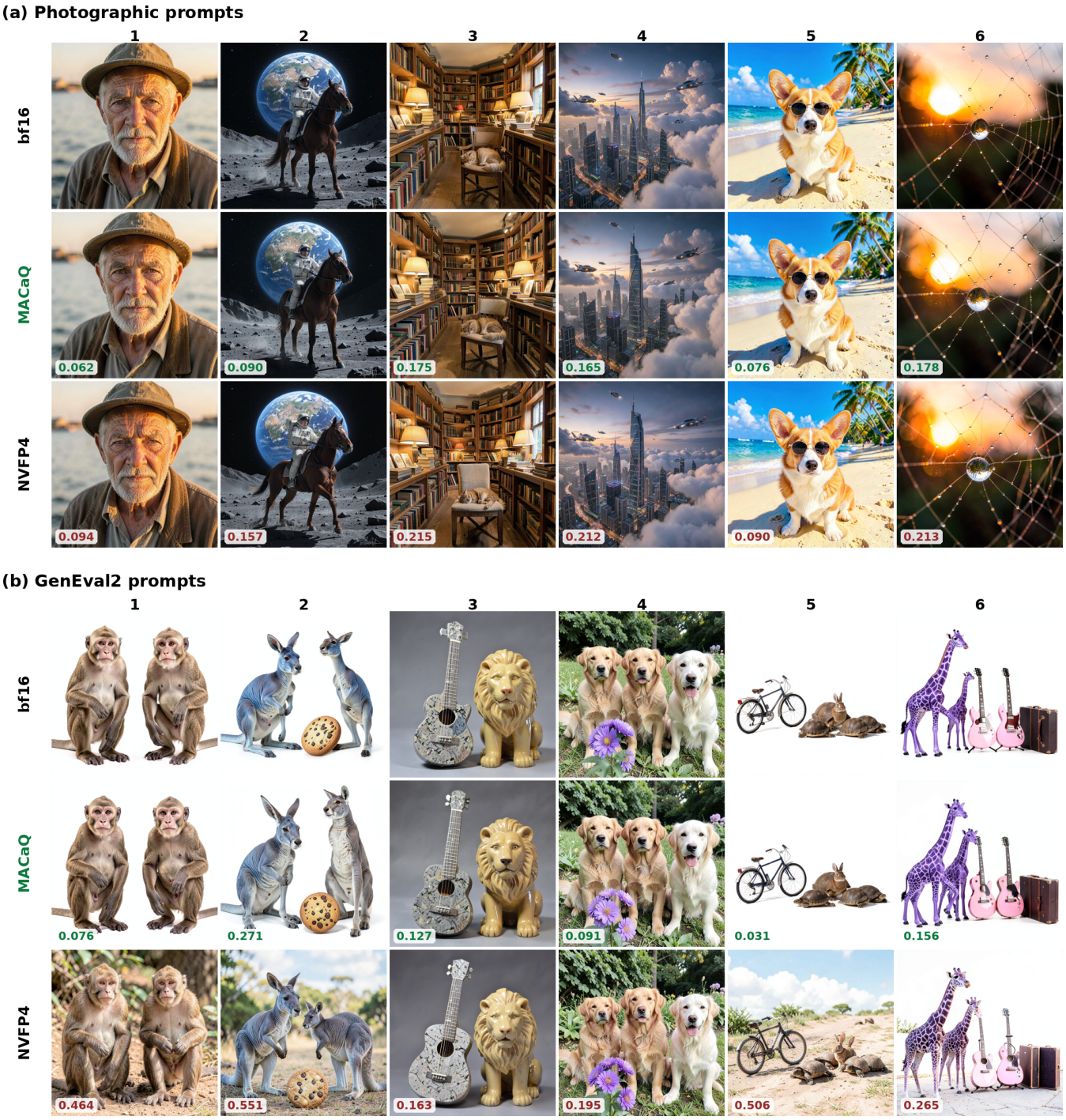

Figure 6. Qualitative comparison on FLUX.2-klein-4B: BF16 vs. MACaQ (MXFP4 W4A4) vs. the official NVFP4 checkpoint, scaled to 512 pix width. Same seed per prompt. The inset number shows the lpips (lower is better). For a quantitative comparison see Table 3. **(a) Photographic prompts:** (1) a photorealistic portrait of an elderly fisherman, weathered face, golden hour light; (2) an astronaut riding a horse on the moon, earth in the background, dramatic; (3) a cozy bookshop interior, warm lamps, stacks of old books, cat sleeping on a chair; (4) a futuristic city skyline at dusk, flying cars, volumetric clouds, highly detailed; (5) a corgi wearing tiny sunglasses on a tropical beach, vibrant colors; (6) a macro photograph of a dewdrop on a spider web at sunrise. **(b) GenEval2 prompts:** (1) four brown monkeys; (2) a blue kangaroo and a cookie; (3) a stone guitar and a plastic lion; (4) two purple flowers in front of three dogs; (5) a bicycle to the left of a rabbit, and five brown turtles; (6) three purple giraffes to the left of two pink guitars, and two suitcases. Best viewed zoomed in.

natural. Completing the square gives

$$\begin{aligned} \text{gain}(i, \mathcal{B}, u) &= \text{const} - (u - u_i^\star)^\top A\,(u - u_i^\star), \\ u_i^\star &= v + A^{-1} g_{i,\mathcal{B}}. \end{aligned} \tag{21}$$

Here $u_i^\star$ is the value the four weights would take if they were not quantized (the exact minimizer over the block with all other weights fixed, i.e. one Newton step on the block), and the best codeword is the one nearest to $u_i^\star$ in the metric defined by A. Block refinement is therefore a nearest-neighbour search: for every row and block, find the admissible codeword closest to that row's target. In the scalar case of the main text this search has two candidates, the adjacent levels, so its cost never mattered. Here it has up to 8192 candidates, and we organize it accordingly.

We score all admissible candidates and accept the best improving move. For `IQ3_S`, this means jointly searching magnitudes and signs over $512 \times 16 = 8192$ signed codewords. For `IQ3_XXS`, we alternate between selecting one of 256 codewords with signs fixed and selecting one of 128 legal sign patterns per eight-weight group with magnitudes fixed. Each search is exact conditional on the quantities held fixed.

Efficient exact scoring. A nearest-neighbour search under a quadratic metric is evaluated the same way as the assignment step of k-means: expand the square. Dropping the terms of Eq. (21) that do not depend on u leaves

$$\begin{aligned} S_i(u) &= \underbrace{u^\top A\, u}_{\text{codeword "length"}} - 2 \underbrace{u^\top q_i}_{\text{match to row } i}, \\ q_i &= Av + g_{i,\mathcal{B}}. \end{aligned} \tag{22}$$

and the codeword with the smallest S_i is the nearest one. The two terms have very different costs. The length term $u^\top Au$ depends on the codeword and on the column block, but not on the row: the metric A is a block of H_f, which is shared by all m output rows of the layer. It is therefore a table with one entry per codeword, computed once per block and reused by every row. The match term is a dot product between the row's 4-vector q_i and the codeword. Computing it for all rows and all candidates at once is a single matrix product, $Q\,\mathcal{C}^\top$, where $Q \in \mathbb{R}^{m\times 4}$ stacks the q_i and $\mathcal{C} \in \mathbb{R}^{C\times 4}$ stacks the candidates. The score matrix $(m \times C)$ is the table broadcast over rows minus twice this product, and a row-wise argmin selects the codewords.

The block scale enters as a per-row scalar: a candidate is $u = s_i c$ with c a signed codeword, so $S_i(s_i c) = s_i^2\, c^\top Ac - 2 s_i\, q_i^\top c$, and the table and the product are combined with per-row factors. The table itself is obtained with one small product per block: with $\text{vec}(cc^\top) \in \mathbb{R}^{16}$ precomputed for every codeword, the C lengths are $\text{vec}(cc^\top) \cdot \text{vec}(A)$, a $(C \times 16)(16)$ product.

For `IQ3_S` the 16 sign patterns of a block are folded into the codebook, so one table of $C = 8192$ lengths serves all rows. For `IQ3_XXS` the signs are held fixed during the magnitude search and differ between rows, so the metric becomes row-dependent, $(\sigma_i \sigma_i^\top) \odot A$: the table is now computed per row, as an $(m \times 16)(16 \times 256)$ product, which is still one GEMM. The sign search over the 128 parity-legal patterns of an 8-group has the same structure with the magnitudes μ_i fixed and the metric $(\mu_i \mu_i^\top) \odot (H_f)_{\mathcal{B},\mathcal{B}}$ on the 8×8 block: an $(m \times 64)(64 \times 128)$ product for the lengths and an $(m \times 8)(8 \times 128)$ product for the matches.

After a block is replaced in a set of rows, the gradients of those rows are corrected exactly with the rank-4 analogue of Eq. (16),

$$g_i \leftarrow g_i - (H_f)_{:,\mathcal{B}}\,(u - v), \tag{23}$$

which adds four Hessian columns at a cost of $4n$ per changed row instead of the n^2 of a fresh $H_f r_i^\top$. Only rows that accepted a move are touched.

Sweeps and cost. Rows are processed in parallel, while blocks are visited sequentially in random order using the updated gradients. We require a relative improvement above 10^{-6} and recompute the gradients once per sweep to limit numerical drift. Refinement stops after at most eight sweeps or when a sweep accepts no move.

Per sweep over the $n/4$ blocks of an `IQ3_S` tensor, the match products amount to $2mnC$ operations in total, the length tables to $O(nC)$, and the gradient updates to at most $O(mn^2)$ (if every row moved at every block; in practice far fewer), the same order as the once-per-sweep gradient refresh. The search thus scales linearly with the codebook size and never re-evaluates a full row objective. For the down projection of Qwen3-4B ($m = 2560$, $n = 9728$) with $C = 8192$, the match products are 4.1×10^{11} operations per sweep. The inner dimension of these products is only four and the blocks are processed sequentially, so peak GPU throughput is not reached; eight sweeps of such a tensor, including the sign and scale stages, take approximately 5–8 s. The FP32 score matrix is the only sizeable intermediate (84 MB at these dimensions); for larger tensors we process rows in chunks to keep it below 600 MB. All score products use FP32 with TF32 disabled, because the score differences between neighbouring candidates can be small. Self-tests

on real GGUF tensors check monotone sweeps, agreement with brute-force block minimization, and bit-exact repacking.

Optional scale refinement. With codes and signs fixed, we can also refine the shared scales. For a group $\mathcal{S}$ reconstructed as az, define $A_{\mathcal{S}} = (H_f)_{\mathcal{S},\mathcal{S}}$ and $q = A_{\mathcal{S}} v + g_{i,\mathcal{S}}$. Up to a constant, the objective is

$$L(a) = a^2 z^\top A_{\mathcal{S}} z - 2a\, z^\top q. \tag{24}$$

We evaluate all 16 sub-block scale codes. For the super-block scale, z includes the sub-block factors, and we round the continuous minimizer

$$d^\star = \frac{z^\top q}{z^\top A_{\mathcal{S}} z} \tag{25}$$

to FP16. Scale changes are accepted only when they reduce the objective. When enabled, these two scale stages follow the code and sign searches in each sweep.

Setup. We quantize Qwen3-4B and Qwen3-14B with `llama.cpp` after applying MACaQ's scaling and block-Hadamard rotation. The corresponding activation transform runs online in a patched `llama.cpp`. Hessians are estimated from 10^6 WikiText-2 training tokens, transformed into the quantization basis, and regularized with $\lambda = 1$. The output head uses the Hessian of the final-normalization output.

We compare the initial quantization, scalar flips restricted to legal on-grid moves with $K = 512$, and eight sweeps of block refinement. Initial files are quantized either with or without `llama.cpp`'s importance matrix, computed from the same calibration text. Perplexity is measured with `llama-perplexity` on WikiText-2 and a Pile sample; these measurements are not directly comparable to Tab. 1.

Results. Block refinement is most effective without an importance matrix (Tab. 8). On Qwen3-4B in plain `IQ3_S`, it reduces WikiText-2 perplexity from 13.06 to 11.81, compared with 12.31 for scalar flips and 11.83 for importance-matrix initialization alone. On Qwen3-4B, block refinement reduces individual tensors' Hessian objectives by 30–42%, versus 16–32% for scalar flips, and takes approximately 5–8 seconds per large tensor.

With importance-matrix initialization, the gains depend on the format. Refinement changes `IQ3_S` perplexity only slightly, but improves `IQ3_XXS` on both models. Among the refinements, block search performs best at 4B, while scalar flips perform best at 14B. Exact block minimization thus does not guarantee better model perplexity: the searches can reach different local minima, and the layer objective is only a proxy for model quality.

Scale refinement further reduces the calibration objective but does not improve perplexity over codes-only refinement in the 4B comparisons. We therefore recommend codes-only block refinement for these formats.

Table 8. Weight-only quantization of MACaQ-transformed Qwen3 models. Lower perplexity is better. *imatrix* denotes initialization with `llama.cpp`'s importance matrix; *plain* omits it. *codes* refines codewords and signs only, while *codes+scales* additionally refines the shared scales. Codes-only refinement was not run at 14B.

Format	Refinement	Wiki2	Pile
Qwen3-4B			
F16	–	11.56	7.41
`IQ3_S` (plain)	none	13.06	8.10
	scalar flips	12.31	7.85
	block, codes	**11.81**	**7.67**
	block, codes+scales	11.86	7.68
`IQ3_S` (imatrix)	none	11.83	7.73
	scalar flips	11.91	7.79
	block, codes	11.88	**7.72**
	block, codes+scales	11.89	7.73
`IQ3_XXS` (imatrix)	none	13.09	8.36
	scalar flips	12.76	8.21
	block, codes	**12.64**	**8.14**
	block, codes+scales	12.82	8.21
Qwen3-14B			
F16	–	7.52	5.63
`IQ3_S` (plain)	none	7.75	5.79
	scalar flips	7.74	5.80
	block, codes+scales	**7.68**	**5.75**
`IQ3_S` (imatrix)	none	7.69	5.73
	scalar flips	**7.61**	**5.71**
	block, codes+scales	7.64	5.73
`IQ3_XXS` (imatrix)	none	8.07	5.96
	scalar flips	**7.92**	**5.90**
	block, codes+scales	7.98	5.95